

Predicting Urban NO₂ Concentrations from Low-Cost Sensor Arrays

STAT 24620 / FINM 34700 / STAT 32950 — Group Data Project, Spring 2026

[Name 1], [Name 2], [Name 3]

May 11, 2026

AI acknowledgment: Claude Code (Anthropic) was used for coding assistance and debugging. All analysis design, interpretation, and written explanations are the authors' own work.

Research Question

Can a five-sensor electrochemical array (PT08_S1–S5) accurately predict hourly NO₂ concentrations at an Italian urban monitoring site, and does dimensionality reduction via PCA improve prediction relative to directly regressing on the raw sensor readings? Air-quality monitoring increasingly relies on low-cost sensors alongside certified reference analyzers. If a small number of principal components captures most sensor variance, PC regression may be more stable than full sensor regression — especially when sensors are collinear and subject to environmental drift. We compare OLS, Ridge, Lasso, and PC regression, all augmented with meteorological covariates, on a chronological hold-out test set.

Dataset

Source: UCI Machine Learning Repository — Air Quality Dataset (De Vito et al., 2008). Hourly measurements from March 2004 to April 2005 at one urban site in Italy. The project-provided CSV was partially pre-cleaned: heavily missing variables were dropped, rows without valid dates were removed, and remaining gaps were median-imputed. Residual −200 sentinel codes survive in CO_GT and C6H6_GT; these two variables are not used as predictors or as the response, so they do not affect any model results. **Variable groups:** *Sensors* PT08_S1_CO, PT08_S2_NMHC, PT08_S3_NOx, PT08_S4_NO2, PT08_S5_O3; *weather* T (°C), RH (%), AH (g/m³); *reference* CO_GT, C6H6_GT, NOx_GT, NO2_GT; *response* NO2_GT (μg/m³).

Section 1: EDA and Preprocessing

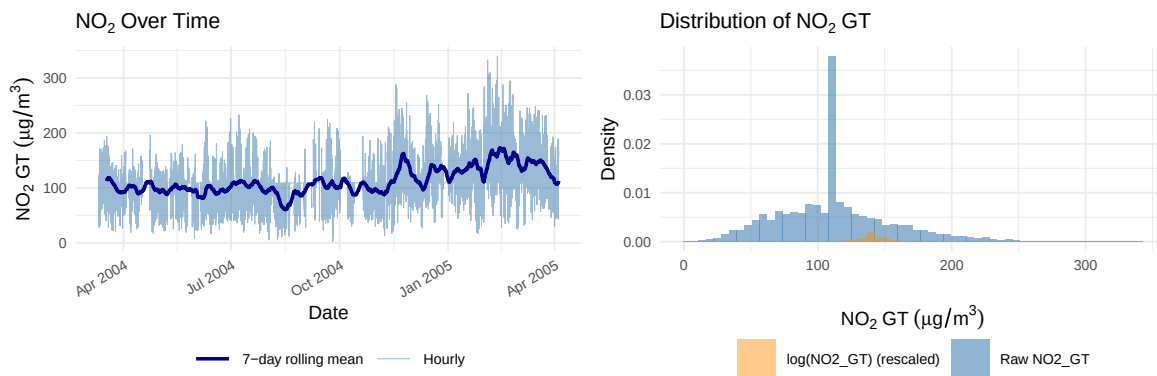


Figure 1: NO₂ reference concentration over time (left) and distribution on raw vs. log scale (right).

The time-series panel shows strong diurnal cycling (morning and evening traffic peaks) and a seasonal drift: concentrations rise in autumn/winter when boundary-layer mixing is suppressed and fall in spring/summer. Raw

NO2_GT is mildly right-skewed (0.73), while the log transform over-corrects to a left-skewed distribution (-1.09). Since the log transform does not improve symmetry, we fit all supervised models on the raw concentration scale and note the choice as a limitation.

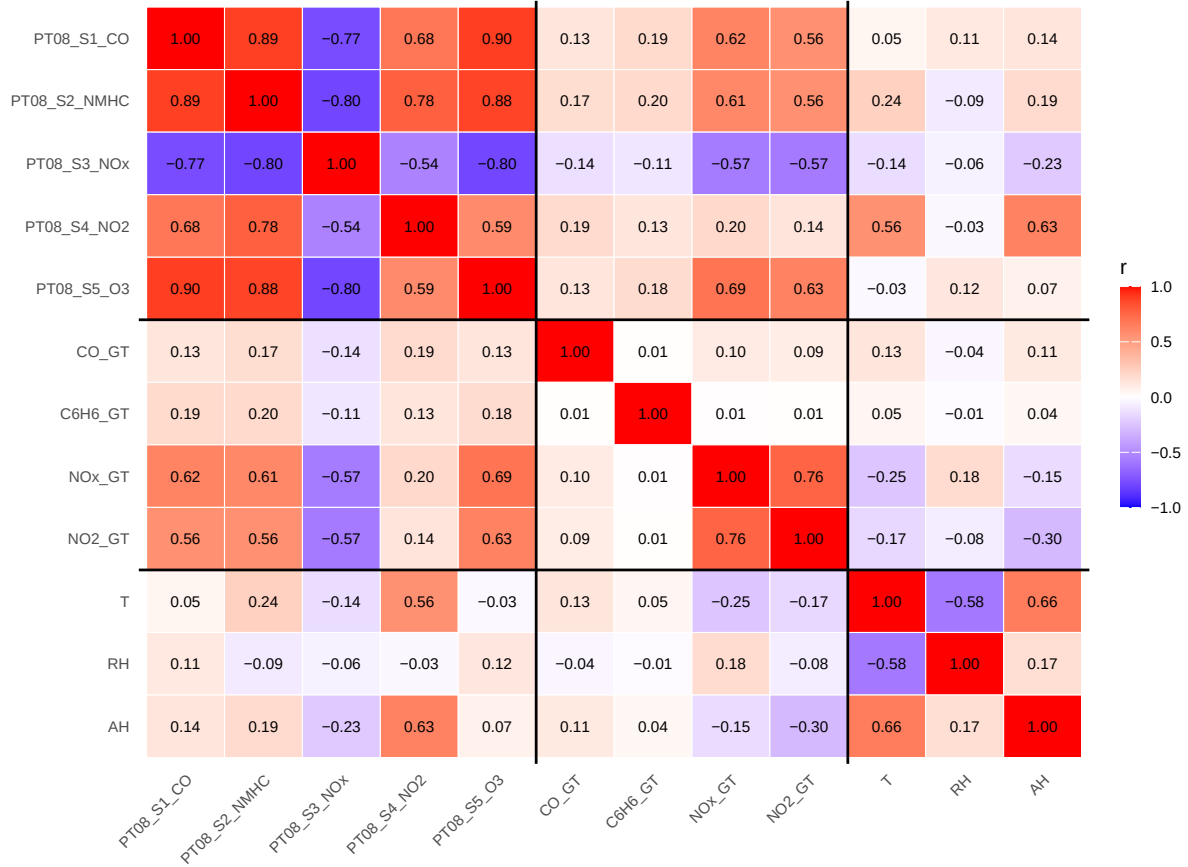


Figure 2: Pearson correlation heatmap across sensors (top-left block), reference analyzers (middle block), and weather (bottom-right block). Block separators are drawn in black.

The block structure motivates both PCA (the sensor block is internally correlated, suggesting low intrinsic rank) and the supervised analysis (cross-block correlation between sensors and NO2_GT supports sensor-based prediction).

Section 2: Chronological Train / Test Split

Because the data are hourly time-series with strong autocorrelation and seasonal drift, a random split would leak future information into the past. We use a chronological holdout: the first 75% form the training set, the remaining 25% form the test set. All preprocessing statistics (scaling means/SDs, PCA directions, regularization hyperparameters) are estimated on training data only and applied to test data without re-fitting.

Training: 7017 rows (2004-03-10 to 2004-12-28). Test: 2340 rows (2004-12-28 to 2005-04-04). Split at

Section 3: PCA on the Sensor Block

We run PCA exclusively on the five PT08 sensor variables. Weather variables are excluded so that the structural story stays clean: the question here is *how many independent axes of variation exist in the sensor array itself?* Weather enters only as covariates in the supervised stage. Standardization (zero mean, unit variance) uses training-set statistics only and is applied identically to the test sensors.

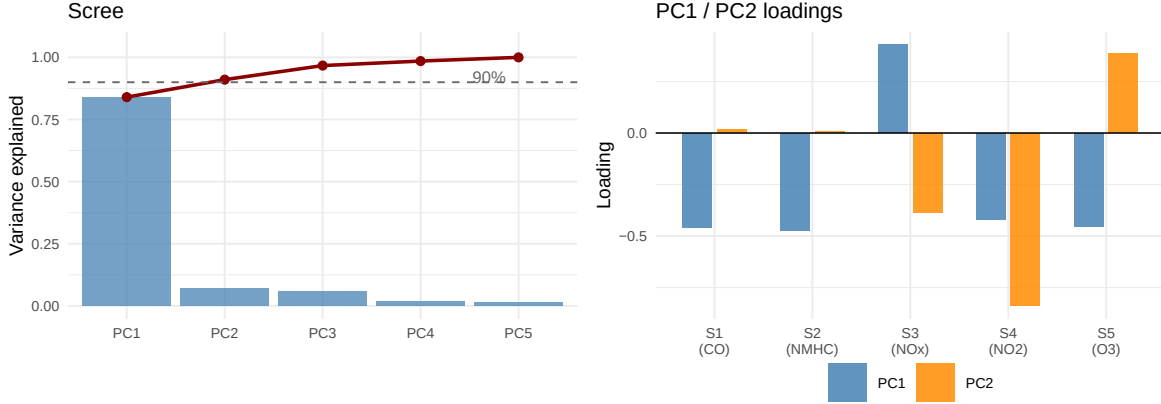


Figure 3: Scree plot (left) and PC1/PC2 sensor loadings (right).

PC1 (84% of variance) loads positively and nearly equally on four of the five sensors ($\approx +0.42$ to $+0.47$), but *negatively* on PT08_S3_NOx (-0.43). This sign flip is physically meaningful: PT08_S3 is a metal-oxide sensor tuned to NOx, which responds inversely to ambient NO₂ oxidising conditions compared to the other sensors. PC1 therefore represents the overall combustion/traffic pollution load, with PT08_S3 providing a counteracting signal. **PC2 (7% of variance)** is strongly dominated by PT08_S4_NO2 ($+0.84$), with PT08_S3_NOx ($+0.39$) and PT08_S5_O3 (-0.38) as secondary contributors — reflecting the chemical antagonism between NO₂ and O₃ in urban photochemistry. Together PC1 and PC2 explain 91% of sensor variance; PC3–PC5 each capture less than 6%. The low-rank structure motivates PC regression: projecting onto the first few PCs substitutes correlated raw sensors with orthogonal summary axes, which can stabilise regression coefficients. Whether this leads to better out-of-sample prediction depends on whether the discarded PCs carry useful signal — tested in Section 5.

Section 4: Supervised Models

We fit four models predicting N02_GT (raw scale), each using the five sensors plus T, RH, AH as predictors. All predictors are standardised using a separate scaler fit on training data only. **OLS** (baseline, no regularisation); **Ridge** (L_2 penalty, λ by 5-fold CV); **Lasso** (L_1 penalty, λ by 5-fold CV); **PC regression** (OLS on first k PCs + weather, k by 5-fold CV). This R implementation uses `glmnet`, whose penalty λ is parameterised differently from `scikit-learn`’s α (`glmnet` scales the loss by $1/n$); optimal numerical values therefore differ from a Python implementation, but the methodology is identical.

Section 5: Comparison

Table 1: Model comparison on the chronological hold-out test set.

Model	Train RMSE	Test RMSE	Test R^2
OLS (baseline)	25.169	40.492	0.345
Ridge (lambda=2.466)	25.324	40.746	0.337
Lasso (lambda=0.0334)	25.172	40.437	0.347
PC Regression (k=5)	25.169	40.492	0.345

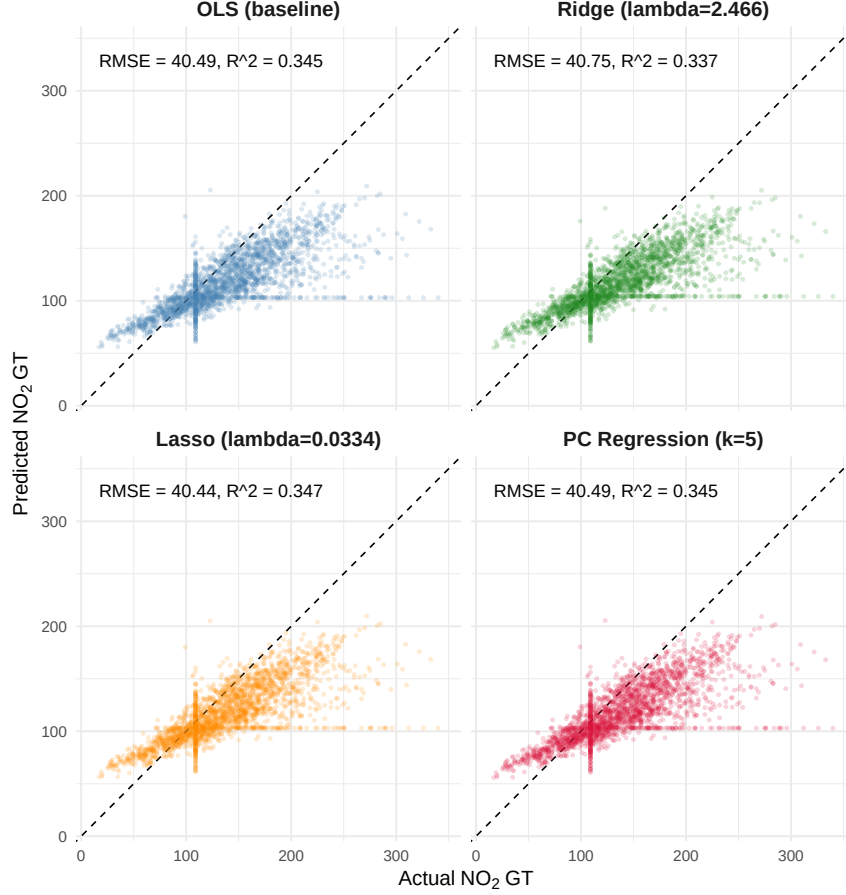


Figure 4: Predicted vs. actual NO_2 on the chronological test set, by model.

Discussion. All four models achieve nearly identical test performance ($\text{RMSE} \sim 40 \text{ } \mu\text{g}/\text{m}^3$, $R^2 \approx 0.35$), and all show a large train-to-test RMSE gap (~ 25 train vs. ~ 40 test). The gap is larger than typical overfitting: the training set covers spring through autumn 2004, while the test set is winter 2004–2005. Seasonal changes in boundary-layer dynamics, photochemistry, and sensor behaviour mean the test distribution is systematically different from training. **OLS** is not badly overfit relative to the regularised alternatives, suggesting the eight predictors are informative but the main challenge is distributional shift, not in-sample variance. **Ridge** (CV-optimal $\lambda = 2.466$) gives marginal improvement over OLS, confirming that coefficient instability from multicollinearity is not the binding constraint. **Lasso** (CV-optimal $\lambda = 0.0334$) retains 8 of 8 predictors — noteworthy because PC1 alone explains 84% of sensor variance, yet Lasso assigns nonzero weight to most sensors individually. Each retained sensor contributes marginal information about NO_2GT beyond what the others provide; PT08_S3_NOx (negative on PC1) is retained, confirming it adds predictive signal the variance-maximising PC1 direction partially suppresses. **PC Regression** ($k = 5$) shows CV RMSE decreasing monotonically from $k = 1$ to $k = 5$, meaning every PC — including PC3–PC5, which together explain only 9% of sensor variance — contributes useful information about NO_2GT . With $k = 5$, PC regression is algebraically equivalent to OLS on the scaled raw sensors (it spans the same column space), explaining identical test metrics. This is the core tension between PCA and Lasso: PCA discards low-variance PCs on structural grounds, but CV reveals those directions are predictively relevant.

Section 6: Limitations

1. **Residual autocorrelation.** Hourly observations are strongly serially correlated. OLS standard errors assume independence, which they do not have. Newey–West (HAC) standard errors would correct inference without changing point estimates; this was not implemented. The CV folds also do not account for serial correlation, so reported CV RMSEs are optimistic.

2. **Seasonal distributional shift / sensor drift.** The large train-test RMSE gap (~ 25 vs. ~ 40) is not explained by overfitting alone — it reflects systematic differences between training (spring–autumn 2004) and test (winter 2004–2005). Electrochemical sensors are known to drift over months of operation and respond differently under winter atmospheric chemistry.
3. **Median imputation compresses tails.** The provided CSV replaced missing values with column medians. Imputing at the median artificially pulls extreme observations toward the centre, understating tail variance.
4. **Single monitoring site.** All data come from one urban location in Italy over one year. Results may not generalise to other cities, sensor deployments, climates, or traffic profiles.
5. **Reference measurement error.** GT variables carry calibration uncertainty and sampling noise. Treating `N02_GT` as noiseless ground truth introduces errors-in-variables bias — true predictive signal from the sensors may be somewhat higher than R^2 values suggest.
6. **Log transform omitted.** Raw `N02_GT` is mildly right-skewed (skewness 0.73), but the log transform over-corrects to a left-skewed distribution (skewness -1.09). A Box–Cox transformation could achieve better symmetry but was not explored.

Code Appendix

All code used to produce the results above is reproduced here in the order it appears in the report. Chunks are not re-executed; this appendix is for inspection and reproducibility.

```
# Global knitr options: hide code throughout the body; show results, figures, tables.
# All code is reproduced in the Code Appendix at the end.
knitr::opts_chunk$set(
  echo      = FALSE,
  message   = FALSE,
  warning    = FALSE,
  fig.align = "center",
  fig.width  = 6.5,
  fig.height = 3,
  dpi        = 120,
  out.width  = "92%"
)

# Packages
library(tidyverse)  # dplyr, ggplot2, tidyr, readr
library(lubridate)  # datetime arithmetic
library(patchwork)  # combining ggplots
library(glmnet)     # ridge + lasso
library(e1071)      # skewness()
library(zoo)        # rolling means
library(knitr)      # kable()

set.seed(42)
df <- read.csv("C:/Users/MiahH/OneDrive/Documents/FINM-34700-HWs/air_quality_italy.csv",
  stringsAsFactors = FALSE)

# Diagnostic checks (suppressed in body; visible in Code Appendix)
cat("Shape:", nrow(df), "rows x", ncol(df), "cols\n")
print(sapply(df, class))
print(colSums(is.na(df)))

num_cols <- names(df)[sapply(df, is.numeric)]
neg200_hit <- sapply(df[num_cols], function(x) any(x == -200, na.rm = TRUE))
neg200_in <- names(neg200_hit)[neg200_hit]
if (length(neg200_in)) print(neg200_in) else cat("None -- clean.\n")

# Build proper datetime index
df$timestamp <- as.POSIXct(df$Date, tz = "UTC") + lubridate::hours(df$Hour)
df <- df[order(df$timestamp), ]
rownames(df) <- NULL

cat("\nDate range:", format(min(df$timestamp)), "to", format(max(df$timestamp)), "\n")
cat("Total hours:", nrow(df), "\n")
df$NO2_roll <- zoo::rollmean(df$NO2_GT, k = 168, fill = NA, align = "right")

p_ts <- ggplot(df, aes(x = timestamp)) +
  geom_line(aes(y = NO2_GT, color = "Hourly"), linewidth = 0.2, alpha = 0.55) +
  geom_line(aes(y = NO2_roll, color = "7-day rolling mean"), linewidth = 0.7) +
  scale_color_manual(name = NULL,
    values = c("Hourly" = "steelblue",
      "7-day rolling mean" = "darkblue")) +
```

```

labs(x = "Date", y = expression(NO[2]~GT~(mu*g/m^3)),
     title = expression(NO[2]~"Over Time")) +
theme_minimal(base_size = 8) +
theme(legend.position = "bottom",
      legend.margin = margin(0,0,0,0),
      legend.box.spacing = unit(2, "pt"),
      axis.text.x = element_text(angle = 30, hjust = 1),
      plot.title = element_text(size = 9))

log_vals <- log(df$NO2_GT[df$NO2_GT > 0 & !is.na(df$NO2_GT)])

p_dist <- ggplot() +
  geom_histogram(aes(x = df$NO2_GT, y = after_stat(density), fill = "Raw NO2_GT"),
                bins = 60, alpha = 0.6) +
  geom_histogram(aes(x = log_vals * 30, y = after_stat(density) / 30,
                    fill = "log(NO2_GT) (rescaled)"),
                bins = 60, alpha = 0.45) +
  scale_fill_manual(name = NULL,
                    values = c("Raw NO2_GT" = "steelblue",
                              "log(NO2_GT) (rescaled)" = "darkorange")) +
  labs(x = expression(NO[2]~GT~(mu*g/m^3)), y = "Density",
       title = expression("Distribution of"~NO[2]~GT)) +
  theme_minimal(base_size = 8) +
  theme(legend.position = "bottom",
        legend.margin = margin(0,0,0,0),
        legend.box.spacing = unit(2, "pt"),
        plot.title = element_text(size = 9))

p_ts + p_dist

SENSORS <- c("PT08_S1_CO", "PT08_S2_NMHC", "PT08_S3_NOx", "PT08_S4_NO2", "PT08_S5_O3")
GT_VARS <- c("CO_GT", "C6H6_GT", "NOx_GT", "NO2_GT")
WEATHER <- c("T", "RH", "AH")
TARGET <- "NO2_GT"

corr_cols <- c(SENSORS, GT_VARS, WEATHER)
corr_mat <- cor(df[, corr_cols], use = "pairwise.complete.obs")

corr_df <- as.data.frame(as.table(corr_mat))
names(corr_df) <- c("Var1", "Var2", "value")
corr_df$Var1 <- factor(corr_df$Var1, levels = corr_cols)
corr_df$Var2 <- factor(corr_df$Var2, levels = rev(corr_cols))

block_breaks <- c(length(SENSORS), length(SENSORS) + length(GT_VARS))

ggplot(corr_df, aes(Var1, Var2, fill = value)) +
  geom_tile(color = "white", linewidth = 0.2) +
  geom_text(aes(label = sprintf("%.2f", value)), size = 2.1) +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red",
                      midpoint = 0, limits = c(-1, 1), name = "r") +
  geom_vline(xintercept = block_breaks + 0.5, linewidth = 0.5) +
  geom_hline(yintercept = length(corr_cols) - block_breaks + 0.5, linewidth = 0.5) +
  labs(x = NULL, y = NULL) +
  theme_minimal(base_size = 8) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        panel.grid = element_blank(),

```

```

        legend.key.width = unit(8, "pt"))
n      <- nrow(df)
split_idx <- floor(0.75 * n)
train <- df[1:split_idx, ]
test  <- df[(split_idx + 1):n, ]

cat(sprintf("Training: %d rows (%s to %s). Test: %d rows (%s to %s). Split at %s.\n",
            nrow(train),
            format(min(train$timestamp), "%Y-%m-%d"),
            format(max(train$timestamp), "%Y-%m-%d"),
            nrow(test),
            format(min(test$timestamp), "%Y-%m-%d"),
            format(max(test$timestamp), "%Y-%m-%d"),
            format(min(test$timestamp), "%Y-%m-%d")))
train_sensors <- as.matrix(train[, SENSORS])
test_sensors  <- as.matrix(test[, SENSORS])

sens_mean <- colMeans(train_sensors)
sens_sd   <- apply(train_sensors, 2, sd)

train_sensors_sc <- scale(train_sensors, center = sens_mean, scale = sens_sd)
test_sensors_sc  <- scale(test_sensors,  center = sens_mean, scale = sens_sd)

pca <- prcomp(train_sensors_sc, center = FALSE, scale. = FALSE)
train_pcs <- pca$x
test_pcs  <- as.matrix(test_sensors_sc) %*% pca$rotation

var_exp <- pca$sdev^2 / sum(pca$sdev^2)
cum_var <- cumsum(var_exp)

loadings <- pca$rotation
colnames(loadings) <- paste0("PC", 1:ncol(loadings))

# Suppressed; figure and discussion convey the same information
cat("Variance explained per PC:\n")
for (i in seq_along(var_exp)) {
  cat(sprintf(" PC%d: %.3f (cumulative: %.3f)\n", i, var_exp[i], cum_var[i]))
}
cat("\nLoadings matrix:\n")
print(round(loadings, 3))
screedf <- data.frame(PC = factor(paste0("PC", 1:5), levels = paste0("PC", 1:5)),
                      var = var_exp, cum = cum_var)

p_screedf <- ggplot(screedf, aes(x = PC)) +
  geom_col(aes(y = var), fill = "steelblue", alpha = 0.75) +
  geom_line(aes(y = cum, group = 1), color = "darkred", linewidth = 0.7) +
  geom_point(aes(y = cum), color = "darkred", size = 1.5) +
  geom_hline(yintercept = 0.9, linetype = "dashed", color = "gray40") +
  annotate("text", x = 4.7, y = 0.92, label = "90%", color = "gray40", size = 2.5) +
  scale_y_continuous(limits = c(0, 1.05)) +
  labs(y = "Variance explained", title = "Scree", x = NULL) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(size = 9))

sensor_labels <- c(

```



```

"PT08_S1_CO" = "S1\n(CO)",
"PT08_S2_NMHC" = "S2\n(NMHC)",
"PT08_S3_NOx" = "S3\n(NOx)",
"PT08_S4_NO2" = "S4\n(NO2)",
"PT08_S5_O3" = "S5\n(O3)"
)

load_df <- as.data.frame(loadings[, 1:2])
load_df$sensor <- factor(rownames(load_df), levels = SENSORS)
load_long <- tidyr::pivot_longer(load_df, cols = c("PC1", "PC2"),
                                names_to = "component", values_to = "loading")

p_load <- ggplot(load_long,
                 aes(x = sensor, y = loading, fill = component)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6, alpha = 0.85) +
  geom_hline(yintercept = 0, color = "black", linewidth = 0.3) +
  scale_fill_manual(values = c("PC1" = "steelblue", "PC2" = "darkorange"),
                    name = NULL) +
  scale_x_discrete(labels = sensor_labels) +
  labs(y = "Loading", x = NULL, title = "PC1 / PC2 loadings") +
  theme_minimal(base_size = 8) +
  theme(legend.position = "bottom",
        legend.margin = margin(0,0,0,0),
        legend.box.spacing = unit(2, "pt"),
        plot.title = element_text(size = 9))

p_scree + p_load
FEATURES <- c(SENSORS, WEATHER)

X_train_raw <- as.matrix(train[, FEATURES])
X_test_raw <- as.matrix(test[, FEATURES])

sup_mean <- colMeans(X_train_raw)
sup_sd <- apply(X_train_raw, 2, sd)

X_train <- scale(X_train_raw, center = sup_mean, scale = sup_sd)
X_test <- scale(X_test_raw, center = sup_mean, scale = sup_sd)

y_train <- train[[TARGET]]
y_test <- test[[TARGET]]
# --- OLS ---
ols_df_train <- data.frame(y = y_train, X_train)
ols_fit <- lm(y ~ ., data = ols_df_train)
ols_train_pred <- predict(ols_fit, newdata = ols_df_train)
ols_test_pred <- predict(ols_fit, newdata = data.frame(X_test))

# --- Ridge (5-fold CV on train) ---
ridge_cv <- cv.glmnet(X_train, y_train, alpha = 0, nfolds = 5,
                      type.measure = "mse", standardize = FALSE)
ridge_lambda <- ridge_cv$lambda.min
ridge_train_pred <- as.numeric(predict(ridge_cv, newx = X_train, s = "lambda.min"))
ridge_test_pred <- as.numeric(predict(ridge_cv, newx = X_test, s = "lambda.min"))

# --- Lasso (5-fold CV on train) ---
lasso_cv <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 5,

```

```

        type.measure = "mse", standardize = FALSE)
lasso_lambda <- lasso_cv$lambda.min
lasso_train_pred <- as.numeric(predict(lasso_cv, newx = X_train, s = "lambda.min"))
lasso_test_pred <- as.numeric(predict(lasso_cv, newx = X_test, s = "lambda.min"))

lasso_coefs_full <- as.numeric(coef(lasso_cv, s = "lambda.min"))
names(lasso_coefs_full) <- c("(Intercept)", FEATURES)
lasso_coefs <- lasso_coefs_full[-1]

zeroed <- names(lasso_coefs)[lasso_coefs == 0]
retained <- names(lasso_coefs)[lasso_coefs != 0]
# --- PC Regression: choose k by 5-fold CV on train (preserving time order) ---
train_weather_raw <- as.matrix(train[, WEATHER])
test_weather_raw <- as.matrix(test[, WEATHER])
w_mean <- colMeans(train_weather_raw)
w_sd <- apply(train_weather_raw, 2, sd)
train_weather_sc <- scale(train_weather_raw, center = w_mean, scale = w_sd)
test_weather_sc <- scale(test_weather_raw, center = w_mean, scale = w_sd)

n_train <- nrow(X_train)
fold_id <- ceiling(seq_len(n_train) / (n_train / 5))
fold_id <- pmin(fold_id, 5L)

cv_rmse <- numeric(5)
for (k in 1:5) {
  fold_rmse <- numeric(5)
  X_pc_full <- cbind(train_pcs[, 1:k, drop = FALSE], train_weather_sc)
  for (f in 1:5) {
    val_idx <- which(fold_id == f)
    tr_idx <- setdiff(seq_len(n_train), val_idx)
    fit <- lm.fit(cbind(1, X_pc_full[tr_idx, , drop = FALSE]), y_train[tr_idx])
    pred <- cbind(1, X_pc_full[val_idx, , drop = FALSE]) %*% fit$coefficients
    fold_rmse[f] <- sqrt(mean((y_train[val_idx] - pred)^2))
  }
  cv_rmse[k] <- mean(fold_rmse)
}

best_k <- which.min(cv_rmse)

X_pc_train_best <- cbind(train_pcs[, 1:best_k, drop = FALSE], train_weather_sc)
X_pc_test_best <- cbind(test_pcs[, 1:best_k, drop = FALSE], test_weather_sc)
colnames(X_pc_train_best) <- c(paste0("PC", 1:best_k), WEATHER)
colnames(X_pc_test_best) <- colnames(X_pc_train_best)

pcr_df_train <- data.frame(y = y_train, X_pc_train_best)
pcr_fit <- lm(y ~ ., data = pcr_df_train)
pcr_train_pred <- predict(pcr_fit, newdata = pcr_df_train)
pcr_test_pred <- predict(pcr_fit, newdata = data.frame(X_pc_test_best))
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
r2 <- function(y, yhat) 1 - sum((y - yhat)^2) / sum((y - mean(y))^2)

results <- data.frame(
  Model = c(
    "OLS (baseline)",
    sprintf("Ridge (lambda=%.3f)", ridge_lambda),

```

```

    sprintf("Lasso (lambda=%.4f)", lasso_lambda),
    sprintf("PC Regression (k=%d)", best_k)
  ),
  `Train RMSE` = c(rmse(y_train, ols_train_pred),
                  rmse(y_train, ridge_train_pred),
                  rmse(y_train, lasso_train_pred),
                  rmse(y_train, pcr_train_pred)),
  `Test RMSE` = c(rmse(y_test, ols_test_pred),
                  rmse(y_test, ridge_test_pred),
                  rmse(y_test, lasso_test_pred),
                  rmse(y_test, pcr_test_pred)),
  `Test R2` = c(r2(y_test, ols_test_pred),
                r2(y_test, ridge_test_pred),
                r2(y_test, lasso_test_pred),
                r2(y_test, pcr_test_pred)),
  check.names = FALSE
)
results[, 2:4] <- round(results[, 2:4], 3)

kable(results,
      caption = "Model comparison on the chronological hold-out test set.",
      booktabs = TRUE,
      col.names = c("Model", "Train RMSE", "Test RMSE", "Test  $R^2$ "),
      escape = FALSE)
model_preds <- list(
  list(name = "OLS (baseline)", pred = ols_test_pred, color = "steelblue"),
  list(name = sprintf("Ridge (lambda=%.3f)", ridge_lambda), pred = ridge_test_pred, color = "forestgreen"),
  list(name = sprintf("Lasso (lambda=%.4f)", lasso_lambda), pred = lasso_test_pred, color = "darkorange"),
  list(name = sprintf("PC Regression (k=%d)", best_k), pred = pcr_test_pred, color = "#DC143C")
)

plot_df <- do.call(rbind, lapply(model_preds, function(m) {
  data.frame(model = m$name,
             color = m$color,
             actual = y_test,
             predicted = m$pred,
             stringsAsFactors = FALSE)
}))
plot_df$model <- factor(plot_df$model, levels = sapply(model_preds, function(m) m$name))

ann_df <- plot_df %>%
  group_by(model) %>%
  summarise(rmse_val = sqrt(mean((actual - predicted)^2)),
            r2_val = 1 - sum((actual - predicted)^2) / sum((actual - mean(actual))^2),
            .groups = "drop") %>%
  mutate(label = sprintf("RMSE = %.2f, R^2 = %.3f", rmse_val, r2_val))

lo <- min(c(plot_df$actual, plot_df$predicted)) - 5
hi <- max(c(plot_df$actual, plot_df$predicted)) + 5

ggplot(plot_df, aes(x = actual, y = predicted)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
             color = "black", linewidth = 0.3) +
  geom_point(aes(color = model), alpha = 0.18, size = 0.4, show.legend = FALSE) +
  scale_color_manual(values = setNames(sapply(model_preds, function(m) m$color),

```

```

                                supply(model_preds, function(m) m$name))) +
facet_wrap(~ model, ncol = 2) +
geom_text(data = ann_df,
          aes(x = lo + 5, y = hi - 10, label = label),
          hjust = 0, vjust = 1, size = 2.4, inherit.aes = FALSE) +
coord_fixed(xlim = c(lo, hi), ylim = c(lo, hi)) +
labs(x = expression("Actual"~NO[2]~GT),
     y = expression("Predicted"~NO[2]~GT)) +
theme_minimal(base_size = 8) +
theme(strip.text = element_text(face = "bold", size = 8),
      panel.spacing = unit(4, "pt"))

```